

Code implementation: Distilling Context into Parameters for Time-Series Foundation Models for Transportation Forecasting

This notebook walks through the key code of the project. Since the full codebase is extensive and designed to run on a GPU cluster we highlight this notebook is for demonstration purposes only. Helper functions and smaller components are imported from the `src/` module.

Running the full pipeline

To run the full training and evaluation pipeline end to end follow these steps:

1. Install dependencies

```
uv sync
```

2. Train the hypernetwork

```
uv run python -m src.training.main \
    training.target_mode=teacher \
    training.long_context_steps=2016 \
    training.short_context_steps=288 \
    wandb.enabled=true
```

3. Evaluate

3.1 Using the HyperLoRA setup

```
uv run python -m src.orchestration.main \
    orchestration.checkpoint_path=checkpoints/best_hypernet.pt \
    orchestration.long_history_start_date="2017-04-01"
```

4. Baseline evaluation: vanilla Chronos-2, 288-step context

```
uv run python -m src.evaluation.main --config-name experiment_baseline \
    dataset_cfg=dataset/PEMS-BAY
```

5. Cross-dataset transfer (trained on PEMS-BAY, evaluated on METR-LA)

```
uv run python -m src.orchestration.main \
    dataset_cfg=dataset/PEMS-BAY \
    orchestration.eval_dataset_cfg=dataset/METR-LA \
    orchestration.checkpoint_path=checkpoints/best_hypernet.pt
```

Attributions

This project is inspired by, mainly, two other projects:

- [1] J. Pulido and F. Rodrigues, "Time series foundation models as strong baselines in transportation forecasting: A large-scale benchmark analysis," *arXiv preprint arXiv:2602.24238*, 2026.
- [2] R. Charakorn, E. Cetin, S. Uesaka, and R. T. Lange, "Doc-to-lora: Learning to instantly internalize contexts," *arXiv preprint arXiv:2602.15902*, 2026\

Since both projects have published code we have implemented some of this directly and adapted some of it to fit our project.

From [1] we have re-used `src/utils/utils.py` and `src/utils/metrics.py` for data loading, evaluation loop logic and metrics. Our script `src/evaluation/main.py` contains some code snippets from [1] as well, however a lot of this has been adapted to accomodate the LoRA adapter application.

From [2] we have reused some code related to the perciever network, HyperLoRA generator and LoRA injection (implemented in our scripts `src/training/hypernet.py`, `src/training/perceiver.py`, and `src/training/lora_injection.py`). However, since our domain is much different, theirs being text and

ours being time-series data, much of the implementation is heavily modified and thus more of a concept reuse instead of direct code reuse.

1 Setup

We use PyTorch as the main library for data structures, gradient propagation and network architecture.

The Chronos-2 foundational model is loaded via the native python SDK `chronos`

```
In [ ]: import math
import numpy as np
import pandas as pd
from functools import partial
from operator import attrgetter

import torch
import torch.nn as nn
import torch.nn.functional as F
from einops import rearrange
from einops.layers.torch import EinMix as Mix

from chronos import BaseChronosPipeline, Chronos2Pipeline
from omegaconf import OmegaConf

from src.utils import utils as shared_utils
from src.training.perceiver import RMSNorm
from src.orchestration.context_encoder import ChronosContextEncoder
from src.training.perceiver import RMSNorm, SwiGLU

from src.evaluation.main import run_evaluation
```

Load Chronos-2 and config YAML file

```
In [4]: if torch.cuda.is_available(): # PC
        device = torch.device("cuda")
    elif torch.backends.mps.is_available(): # Mac
        device = torch.device("mps")
    else: # Fallback to CPU
        device = torch.device("cpu")

    pipeline = BaseChronosPipeline.from_pretrained("amazon/chronos-2", device_map=device)
    print(f"Total parameters: {sum(p.numel() for p in pipeline.model.parameters())}")
```

Total parameters: 119477664

```
In [5]: # We use OmegaConf to easily manage our configurations
dataset_cfg = OmegaConf.load("conf/dataset/PEMS-BAY.yaml")
training_cfg = OmegaConf.load("conf/experiment_training.yaml")
cfg = OmegaConf.merge(dataset_cfg, training_cfg)
```

2 Hypernetwork

The hypernetwork maps a long historical context window for a single station into a set of LoRA adapter weights. Context flows through stages:

Long history → Context Encoder (frozen Chronos-2) → Perceiver → HyperLoRA → LoRA weights

2.1 Context Encoder

We use the frozen Chronos-2 encoder to extract hidden states from the long context. We use the first 8 of 12 encoder layers.

```
In [8]: context_pipeline = BaseChronosPipeline.from_pretrained("amazon/chronos-2", device_map=device)

# The full encoder code is in src/orchestration/context_encoder.py
context_encoder = ChronosContextEncoder(context_pipeline, num_encoder_layers=8)
```

2.2 Perceiver

The perceiver compresses a variable-length sequence of encoder hidden states into a fixed set of latent queries via cross-attention. 32 learned latent vectors cross-attend to the context, then 384 output queries are decoded, one per layer $\times$ module $\times$ rank slot. See the technical report for details.

```
In [9]: # Define standard cross-attention and self-attention modules used by the Perceiver blocks
```

```
class CrossAttention(nn.Module):
    """Multi-head cross-attention: queries attend to context key-values."""

    def __init__(self, d_model: int, n_heads: int = 8, dropout: float = 0.0):
        super().__init__()
        assert d_model % n_heads == 0
        self.n_heads = n_heads
        self.head_dim = d_model // n_heads
        self.scale = self.head_dim ** -0.5
        self.q_proj = nn.Linear(d_model, d_model, bias=False)
        self.k_proj = nn.Linear(d_model, d_model, bias=False)
        self.v_proj = nn.Linear(d_model, d_model, bias=False)
        self.o_proj = nn.Linear(d_model, d_model, bias=False)
        self.dropout = nn.Dropout(dropout)

    def forward(self, latents: torch.Tensor, context: torch.Tensor) -> torch.Tensor:
        B, N, _ = latents.shape
        S = context.shape[1]
        q = rearrange(self.q_proj(latents), "b n (h d) -> b h n d", h=self.n_heads)
        k = rearrange(self.k_proj(context), "b s (h d) -> b h s d", h=self.n_heads)
        v = rearrange(self.v_proj(context), "b s (h d) -> b h s d", h=self.n_heads)
        attn = (q @ k.transpose(-2, -1)) * self.scale
        attn = F.softmax(attn, dim=-1)
        attn = self.dropout(attn)
        out = attn @ v
        out = rearrange(out, "b h n d -> b n (h d)")
        return self.o_proj(out)

class SelfAttention(nn.Module):
    """Standard multi-head self-attention."""

    def __init__(self, d_model: int, n_heads: int = 8, dropout: float = 0.0):
        super().__init__()
        assert d_model % n_heads == 0
        self.n_heads = n_heads
        self.head_dim = d_model // n_heads
        self.scale = self.head_dim ** -0.5
        self.q_proj = nn.Linear(d_model, d_model, bias=False)
        self.k_proj = nn.Linear(d_model, d_model, bias=False)
        self.v_proj = nn.Linear(d_model, d_model, bias=False)
        self.o_proj = nn.Linear(d_model, d_model, bias=False)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        B, N, _ = x.shape
        q = rearrange(self.q_proj(x), "b n (h d) -> b h n d", h=self.n_heads)
        k = rearrange(self.k_proj(x), "b n (h d) -> b h n d", h=self.n_heads)
        v = rearrange(self.v_proj(x), "b n (h d) -> b h n d", h=self.n_heads)
        attn = (q @ k.transpose(-2, -1)) * self.scale
        attn = F.softmax(attn, dim=-1)
        attn = self.dropout(attn)
        out = attn @ v
        out = rearrange(out, "b h n d -> b n (h d)")
        return self.o_proj(out)
```

```

In [10]: class PerceiverBlock(nn.Module):
    """One block: cross-attention + self-attention + FFN"""

    def __init__(self, d_model: int, n_heads: int = 8,
                  n_self_attn: int = 1, dropout: float = 0.0, ffn_mult: int = 4):
        super().__init__()
        self.x_attn_norm_latents = RMSNorm(d_model)
        self.x_attn_norm_context = RMSNorm(d_model)
        self.x_attn = CrossAttention(d_model, n_heads, dropout)
        self.x_attn_post_norm = RMSNorm(d_model)
        self.self_attn_layers = nn.ModuleList()
        for _ in range(n_self_attn):
            self.self_attn_layers.append(nn.ModuleDict({
                "norm": RMSNorm(d_model),
                "attn": SelfAttention(d_model, n_heads, dropout),
                "post_norm": RMSNorm(d_model),
            }))
        self.ffn_pre_norm = RMSNorm(d_model)
        self.ffn = SwiGLU(d_model, d_model * ffn_mult, d_model)
        self.ffn_post_norm = RMSNorm(d_model)

    def forward(self, latents: torch.Tensor, context: torch.Tensor) -> torch.Tensor:
        residual = latents
        latents = self.x_attn_norm_latents(latents)
        ctx = self.x_attn_norm_context(context)
        latents = self.x_attn_post_norm(self.x_attn(latents, ctx))
        latents = residual + latents
        for sa in self.self_attn_layers:
            residual = latents
            latents = sa["norm"](latents)
            latents = sa["post_norm"](sa["attn"](latents))
            latents = residual + latents
        residual = latents
        latents = self.ffn_pre_norm(latents)
        latents = self.ffn_post_norm(self.ffn(latents))
        latents = residual + latents
        return latents

class PerceiverAggregator(nn.Module):
    """
    PerceiverBlock has an encoder stack followed by a decoder that
    produces the output queries for the HyperLoRA heads.
    """

    def __init__(self, d_input: int, d_latent: int, n_latent_queries: int,
                  n_output_queries: int, num_blocks: int = 2,
                  n_self_attn_per_block: int = 1, n_heads: int = 8,
                  dropout: float = 0.0):
        super().__init__()
        self.d_input = d_input
        self.d_latent = d_latent
        self.n_output_queries = n_output_queries
        self.input_proj = SwiGLU(d_input, d_input * 4, d_latent)
        self.latents_q = nn.Parameter(torch.randn(n_latent_queries, d_latent) * 0.02)
        self.encoder_blocks = nn.ModuleList([
            PerceiverBlock(d_latent, n_heads, n_self_attn_per_block, dropout)
            for _ in range(num_blocks)
        ])
        self.output_queries = nn.Parameter(
            torch.randn(n_output_queries, d_latent) * 0.02
        )
        self.decoder_block = PerceiverBlock(
            d_latent, n_heads, n_self_attn=0, dropout=dropout
        )
        self.final_norm = RMSNorm(d_latent)

    def forward(self, context: torch.Tensor) -> torch.Tensor:
        B = context.shape[0]
        context = self.input_proj(context)
        latents = self.latents_q.unsqueeze(0).expand(B, -1, -1)

```

```

for block in self.encoder_blocks:
    latents = block(latents, context)
out_q = self.output_queries.unsqueeze(0).expand(B, -1, -1)
out = self.decoder_block(out_q, latents)
out = self.final_norm(out)
return out

```

2.3 HyperLoRA generator

Takes the Perceiver's 384 output vectors, passes them through a residual MLP, then through a layer/module-specific linear head that outputs the two LoRA matrices for each layer and module. A small initialization prevents wild initial outputs.

```

In [11]: TARGET_MODULES = ("q", "k", "v", "o")
NUM_LAYERS = 12
D_MODEL = 768

class ResMLPBlock(nn.Module):
    """Residual MLP block for pre-head processing."""

    def __init__(self, d: int, d_hidden: int, dropout: float = 0.0):
        super().__init__()
        self.mlp = nn.Sequential(
            nn.LayerNorm(d),
            nn.Dropout(dropout),
            nn.Linear(d, d_hidden),
            nn.SiLU(),
            nn.Dropout(dropout),
            nn.Linear(d_hidden, d),
            nn.LayerNorm(d),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return x + self.mlp(x)

class HyperLoRA(nn.Module):
    """Generates LoRA A/B matrices for Chronos-2."""

    def __init__(self, d_input: int = D_MODEL, d_latent: int = 256,
                 lora_rank: int = 8, num_layers: int = NUM_LAYERS,
                 d_model: int = D_MODEL, target_modules=TARGET_MODULES,
                 n_latent_queries: int = 64, num_perceiver_blocks: int = 2,
                 n_self_attn_per_block: int = 1, n_heads: int = 8,
                 num_pre_head_layers: int = 2, dropout: float = 0.0):
        super().__init__()
        self.num_layers = num_layers
        self.d_model = d_model
        self.lora_rank = lora_rank
        self.target_modules = target_modules
        self.num_modules = len(target_modules)
        self.d_latent = d_latent

        n_output_queries = num_layers * self.num_modules * lora_rank

        self.perceiver = PerceiverAggregator(
            d_input=d_input, d_latent=d_latent,
            n_latent_queries=n_latent_queries,
            n_output_queries=n_output_queries,
            num_blocks=num_perceiver_blocks,
            n_self_attn_per_block=n_self_attn_per_block,
            n_heads=n_heads, dropout=dropout,
        )

        self.pre_head = nn.Sequential(*[
            ResMLPBlock(d_latent, d_latent * 4, dropout)
            for _ in range(num_pre_head_layers)
        ])

```

```

d_lora = d_model + d_model # A: [r, d_model], B: [d_model, r] → 1536

self.head = Mix(
    "bs n_layers n_modules r d_latent -> bs n_layers n_modules r d_lora",
    weight_shape="n_layers n_modules d_latent d_lora",
    bias_shape=None,
    n_layers=num_layers, n_modules=self.num_modules,
    d_latent=d_latent, r=lora_rank, d_lora=d_lora,
)

head_std = 0.5 / math.sqrt(d_latent + d_lora * lora_rank)
nn.init.normal_(self.head.weight, mean=0.0, std=head_std)

self.bias_A = nn.ParameterDict({
    m: nn.Parameter(
        torch.normal(0, 0.2 / (d_model * lora_rank) ** 0.5,
            (num_layers, lora_rank, d_model))
    ) for m in target_modules
})
self.bias_B = nn.ParameterDict({
    m: nn.Parameter(torch.zeros(num_layers, lora_rank, d_model))
    for m in target_modules
})
self.scaler_A = nn.ParameterDict({
    m: nn.Parameter(torch.ones(1, num_layers, lora_rank, 1))
    for m in target_modules
})
self.scaler_B = nn.ParameterDict({
    m: nn.Parameter(torch.zeros(1, num_layers, lora_rank, 1))
    for m in target_modules
})

def forward(self, context: torch.Tensor) -> dict:
    """context: [batch, seq_len, d_input] → LoRA weight dict."""
    emb = self.perceiver(context)
    emb = rearrange(emb, "b (l m r) d -> b l m r d",
        l=self.num_layers, m=self.num_modules, r=self.lora_rank)
    emb = self.pre_head(emb)

    norm = torch.norm(emb, dim=-1, keepdim=True).clamp(min=1e-8)
    emb = emb / norm

    flat = self.head(emb)

    lora_dict = {}
    for i, module in enumerate(self.target_modules):
        module_flat = flat[:, :, i]
        A_raw, B_raw = module_flat.split(self.d_model, dim=-1)

        A = A_raw * self.scaler_A[module] + self.bias_A[module]
        B = B_raw * self.scaler_B[module] + self.bias_B[module]

        lora_dict[module] = {"A": A, "B": rearrange(B, "b l r d -> b l d r")}

    return lora_dict

```

2.4 LoRA injection

Patches `nn.Linear.forward` on Chronos-2's TimeSelfAttention modules (q, k, v, o) across all 12 encoder layers.

```

In [12]: _MODULE_PATHS = {
    "q": "layer.0.self_attention.q",
    "k": "layer.0.self_attention.k",
    "v": "layer.0.self_attention.v",
    "o": "layer.0.self_attention.o",
}

```

```

In [13]: def _lora_forward(x: torch.Tensor, A: torch.Tensor, B: torch.Tensor,
                        scaling: float, original_forward, *args, **kwargs) -> torch.Tensor:
    base_out = original_forward(x, *args, **kwargs)

    x_float = x.to(A.dtype)
    delta = torch.bmm(x_float, A.transpose(-2, -1))
    delta = torch.bmm(delta, B.transpose(-2, -1))
    delta = delta * scaling

    return (base_out + delta).to(base_out.dtype)

def apply_lora_to_model(model: nn.Module, lora_dict: dict,
                      scaling: float = 2.0) -> list:
    """Patch encoder blocks with LoRA for one forward pass."""
    encoder = model.encoder
    patches = []

    for short_name, weights in lora_dict.items():
        if short_name not in _MODULE_PATHS:
            continue
        A_all = weights["A"]
        B_all = weights["B"]
        module_path = _MODULE_PATHS[short_name]

        for layer_idx, block in enumerate(encoder.block):
            module = getattr(module_path)(block)
            original_forward = module.forward
            A = A_all[:, layer_idx]
            B = B_all[:, layer_idx]
            module.forward = partial(
                _lora_forward, A=A, B=B, scaling=scaling,
                original_forward=original_forward,
            )
            patches.append((module, original_forward))

    return patches

def remove_lora(patches: list) -> None:
    """Restore original forward methods after a training step."""
    for module, original_forward in patches:
        module.forward = original_forward

```

3 Training

The hypernetwork is trained to produce LoRA adapters that let a short-context Chronos-2 student match either a full-context teacher or the ground truth. Only the hypernetwork's parameters are optimized.

3.1 Hierarchical length jitter

To prevent the hypernetwork from overfitting to fixed context lengths, Gaussian noise is added hierarchically: first a batch-level mean, then per-sample lengths around that mean. This greatly improved generalization in validation. The reason why we do this in a hierarchical order is to keep batches efficient with minimal patching.

```

In [14]: def _sample_hierarchical_lengths(batch_size: int, base_steps: int,
                                         sigma_outer: float, sigma_inner: float,
                                         min_steps: int, max_steps: int,
                                         quantize_steps: int) -> torch.Tensor:
    """Sample batch-correlated lengths via outer and inner Gaussians."""
    outer_mean = float(base_steps)
    if sigma_outer > 0:
        outer_mean += float(torch.randn(1).item()) * float(sigma_outer)

    lengths = torch.full((batch_size,), outer_mean, dtype=torch.float32)
    if sigma_inner > 0:
        lengths = lengths + torch.randn(batch_size) * float(sigma_inner)

```

```

lengths = torch.round(lengths)
if quantize_steps > 1:
    lengths = torch.round(lengths / float(quantize_steps)) * float(quantize_steps)

lengths = torch.clamp(lengths, min=float(min_steps), max=float(max_steps))
return lengths.to(torch.int64)

# Demo: sample 4 lengths with batch-level and per-sample jitter
_sampled = _sample_hierarchical_lengths(
    batch_size=4, base_steps=2016,
    sigma_outer=320, sigma_inner=96,
    min_steps=1008, max_steps=4032, quantize_steps=1,
)
print("Sampled lengths:", _sampled.tolist())

```

Sampled lengths: [1753, 1846, 1858, 1810]

3.2 Training objectives

For the training of the hypernetwork we experiment with two training objectives.

3.2.1 Teacher distillation

SmoothL1 between student and teacher quantile predictions

```

In [15]: def quantile_kl_divergence(teacher_quantiles: torch.Tensor,
                                     student_quantiles: torch.Tensor) -> torch.Tensor:
    loss = F.smooth_l1_loss(student_quantiles, teacher_quantiles, reduction="none")
    return loss.mean()

```

3.2.2 Ground truth supervision

Combined quantile loss

```

In [16]: def quantile_crps_loss(student_quantiles: torch.Tensor,
                                targets: torch.Tensor,
                                quantile_levels: torch.Tensor) -> torch.Tensor:
    tau = quantile_levels.to(device=student_quantiles.device,
                              dtype=student_quantiles.dtype)
    tau = tau.view(1, 1, -1, 1)

    errors = targets.to(student_quantiles.dtype).unsqueeze(2) - student_quantiles
    pinball = torch.maximum(tau * errors, (tau - 1.0) * errors)

    return (2.0 * pinball).mean()

```

3.3 Training Dataset

Load PEMS-BAY dataset

```

In [19]: try:
    df_long = shared_utils.load_dataset(cfg)
    print(f"Dataset: {df_long.shape[0]} rows, {df_long[cfg.id_column].nunique()} stations")
    print(f"Time range: {df_long[cfg.timestamp_column].min()} -> {df_long[cfg.timestamp_column].max()}")
except FileNotFoundError:
    print("Dataset file not found. Please ensure the dataset is available at the specified path")

```

Loaded data shape: (52116, 325)
Dataset: 16937700 rows, 325 stations
Time range: 2017-01-01 00:00:00 -> 2017-06-30 22:55:00

3.4 Training loop

The class `HypernetTrainer` orchestrates the full training step. Below is the key method — shown here for illustration, not as an executable cell (it is a class method on `HypernetTrainer`).


```
In [20]: # For illustration: class method part of HypernetTrainer
def _student_forward(self, short_contexts, lora_dict, prediction_length):
    B, Q, T = short_contexts.shape
    num_patches = math.ceil(prediction_length / self.output_patch_size)
    flat_ctx = short_contexts.to(self.device).reshape(B * Q, T)

    expanded_lora = {
        m: {"A": w["A"].repeat_interleave(Q, dim=0),
            "B": w["B"].repeat_interleave(Q, dim=0)}
        for m, w in lora_dict.items()
    }

    patches = apply_lora_to_model(self.model, expanded_lora, self.lora_scaling)
    try:
        out = self.model.forward(flat_ctx, num_output_patches=num_patches)
    finally:
        remove_lora(patches)

    qp = out.quantile_preds[:, :, :prediction_length]
    n_q = qp.shape[1]
    return qp.reshape(B, Q, n_q, prediction_length)
```

```
In [21]: # For illustration: class method part of HypernetTrainer
def train_epoch(self, epoch: int) -> float:
    self.hypernetwork.train()
    total_loss = 0.0
    n_batches = 0
    self.optimizer.zero_grad()

    for batch_idx, batch in enumerate(self.train_loader):
        long_ctx = batch["long_context"].to(self.device)
        short_ctx = batch["short_contexts"]
        sample_indices = batch["sample_indices"]
        targets = batch["targets"].to(self.device)
        prediction_length = targets.shape[-1]

        with torch.no_grad():
            ctx_features = self.context_encoder.encode_last_hidden(long_ctx)

        lora_dict = self.hypernetwork(ctx_features)

        student_preds = self._student_forward(short_ctx, lora_dict, prediction_length)

        if self.target_mode == "teacher":
            teacher_preds = self._lookup_cached_teacher_preds(
                self.train_teacher_cache, sample_indices)
            loss = quantile_kl_divergence(teacher_preds.detach(), student_preds)
        else:
            quantile_levels = self._resolve_quantile_levels(
                n_quantiles=student_preds.shape[2], device=student_preds.device)
            loss = quantile_crps_loss(student_preds, targets, quantile_levels)

        scaled_loss = loss / self.grad_accum_steps
        scaled_loss.backward()

        if (batch_idx + 1) % self.grad_accum_steps == 0:
            if self.grad_clip > 0:
                nn.utils.clip_grad_norm_(self.hypernetwork.parameters(), self.grad_clip)
            self.optimizer.step()
            self.scheduler.step()
            self.optimizer.zero_grad()

        total_loss += loss.item()
        n_batches += 1

    return total_loss / max(n_batches, 1)
```

4. Orchestration

The orchestration layer bridges training and evaluation. It takes a trained hypernetwork checkpoint, runs it over station long contexts, and produces LoRA tensors for the evaluation layer.

```
In [22]: from src.orchestration.run import _compute_prediction_times, _DynamicLoraProvider

def _run_single_station_set(
    *,
    cfg,
    pipeline: Chronos2Pipeline,
    hypernetwork,
    context_encoder: ChronosContextEncoder,
    df_long: pd.DataFrame,
    sensor_ids: list[str],
    station_set_name: str,
) -> tuple[dict, dict]:
    """Run adapter generation + evaluation for one station subset."""
    orch = cfg.orchestration
    id_col = cfg.id_column

    subset_df = df_long[df_long[id_col].astype(str).isin(sensor_ids)].copy()
    if subset_df.empty:
        raise ValueError(f"No rows found for station set '{station_set_name}'")

    prediction_times = _compute_prediction_times(cfg, subset_df)
    print(
        f"Station set '{station_set_name}': sensors={len(sensor_ids)}, "
        f"prediction_steps={len(prediction_times)}"
    )

    encode_batch_size = int(getattr(orch, "encode_batch_size", 32))

    dynamic_lora_provider = None
    dynamic_lora_provider = _DynamicLoraProvider(
        cfg=cfg,
        df_long=subset_df,
        hypernetwork=hypernetwork,
        context_encoder=context_encoder,
        sensor_ids=sensor_ids,
        encode_batch_size=encode_batch_size,
    )

    assignment_rows: list[dict] = []

    for sensor_id in sensor_ids:
        for pt in prediction_times:
            assignment_rows.append(
                {
                    id_col: sensor_id,
                    "prediction_time": pt,
                    "adapter_id": sensor_id,
                }
            )

    print(
        f"Prepared {len(sensor_ids)} dynamic adapters "
        f"(fixed long history, station set '{station_set_name}')"
    )

    assignments_df = pd.DataFrame(assignment_rows)

    eval_cfg = OmegaConf.to_container(cfg, resolve=True)
    eval_cfg = OmegaConf.create(eval_cfg)
    OmegaConf.update(
        eval_cfg,
        "evaluation.history_length_steps",
        int(orch.short_context_length_steps),
    )

    print(
        f"\nStarting evaluation for station set '{station_set_name}' with "
        f"short context = {orch.short_context_length_steps} steps, "
        f"assignments = {len(assignments_df)}"
    )
```

```

)

horizon_metrics, runtime_stats = run_evaluation(
    cfg=eval_cfg,
    pipeline=pipeline,
    df_long=subset_df,
    assignments_df=assignments_df,
    dynamic_lora_provider=dynamic_lora_provider,
    return_runtime=True,
)
return horizon_metrics, runtime_stats

```

```

In [23]: from src.orchestration.run import (_resolve_orchestration_dataset_cfg,
                                             _resolve_station_eval_sets,
                                             _infer_output_layer_lora_dims)

def run_orchestration(cfg) -> tuple[dict, dict]:
    """Generate LoRA adapters from a trained hypernetwork and run evaluation"""
    cfg = _resolve_orchestration_dataset_cfg(cfg)
    orch = cfg.orchestration
    shared_utils.set_seed(cfg.seed)

    if torch.cuda.is_available():
        device = "cuda"
    elif torch.backends.mps.is_available():
        device = "mps"
    else:
        device = "cpu"
    print(f"Using device: {device}")

    df_long = shared_utils.load_dataset(cfg)

    pipeline: Chronos2Pipeline = BaseChronosPipeline.from_pretrained(
        "amazon/chronos-2", device_map=device,
    )

    checkpoint_path = str(orch.checkpoint_path)
    print(f"Loading hypernetwork from {checkpoint_path}")
    hypernetwork = torch.load(checkpoint_path, map_location=device, weights_only=False)
    hypernetwork.eval()
    for p in hypernetwork.parameters():
        p.requires_grad = False

    context_model_name = str(cfg.get("context_encoder_model", "amazon/chronos-2"))
    context_pipeline = BaseChronosPipeline.from_pretrained(
        context_model_name, device_map=device,
    )
    num_encoder_layers = cfg.get("context_encoder_num_layers", None)
    if num_encoder_layers is not None:
        num_encoder_layers = int(num_encoder_layers)
    context_encoder = ChronosContextEncoder(
        context_pipeline, num_encoder_layers=num_encoder_layers,
    )
    context_d_model = int(context_pipeline.model.config.d_model)
    max_ctx = context_encoder.max_context_length
    print(
        f"Context encoder loaded: {context_model_name} "
        f"(d_model={context_d_model}, max_context_length={max_ctx}) "
        f", num_encoder_layers={context_encoder.num_encoder_layers}"
    )

    id_col = cfg.id_column
    sensor_ids = sorted(df_long[id_col].dropna().astype(str).unique().tolist())
    print(f"Number of sensors: {len(sensor_ids)}")

    adapter_cfg = getattr(cfg, "adapter", None)
    rank = 8 if adapter_cfg is None else int(adapter_cfg.get("rank", 8))
    alpha = 16 if adapter_cfg is None else int(adapter_cfg.get("alpha", 16))
    output_lora_d_in, output_lora_d_out = _infer_output_layer_lora_dims(pipeline)
    target_modules = None
    if adapter_cfg is not None and adapter_cfg.get("target_modules") is not None:

```

```
target_modules = list(adapter_cfg.target_modules)

station_sets = _resolve_station_eval_sets(cfg, sensor_ids)

all_horizon_metrics: dict[str, dict] = {}
all_runtime_stats: dict[str, dict] = {}

for station_set_name, station_sensor_ids in station_sets:
    horizon_metrics, runtime_stats = _run_single_station_set(
        cfg=cfg,
        pipeline=pipeline,
        hypernetwork=hypernetwork,
        context_encoder=context_encoder,
        df_long=df_long,
        sensor_ids=station_sensor_ids,
        station_set_name=station_set_name,
    )
    all_horizon_metrics[station_set_name] = horizon_metrics
    all_runtime_stats[station_set_name] = runtime_stats

return all_horizon_metrics, all_runtime_stats
```

5. Evaluation

The evaluation layer runs Chronos-2 in a rolling-window fashion over the test period, applying LoRA adapters in-memory via the dynamic runtime and computing metrics per forecast horizon.

5.1 Dynamic LoRA Runtime

Applies per-sample LoRA tensors in a single batched forward pass by patching Chronos-2 TimeSelfAttention projections.

[illegible]

```

        patches.append((module, original_forward))
    return patches

def remove_dynamic_lora(patches: list) -> None:
    for module, original_forward in patches:
        module.forward = original_forward

```

5.2 Metrics

We implement various metrics, shown here are MAE and coverage used for the report

```

In [25]: def MAE(true, pred, mask_value=None):
    if mask_value is not None:
        mask = np.where(true > mask_value, True, False)
        true, pred = true[mask], pred[mask]
    return np.mean(np.absolute(pred - true))

In [26]: def _find_quantile_col(df: pd.DataFrame, q: float) -> str | None:
    for c in df.columns:
        try:
            if abs(float(c) - q) < 1e-8:
                return c
        except Exception:
            continue
    s = str(q)
    return s if s in df.columns else None

def probabilistic_metrics(forecast_df, true_df, id_column="id",
                          timestamp_column="timestamp", target_column="target",
                          lower_q=0.1, upper_q=0.9):
    lower_col = _find_quantile_col(forecast_df, lower_q)
    upper_col = _find_quantile_col(forecast_df, upper_q)
    merged = pd.merge(
        true_df[[id_column, timestamp_column, target_column]],
        forecast_df[[id_column, timestamp_column, lower_col, upper_col]],
        on=[id_column, timestamp_column], how="inner",
    )
    y_true = merged[target_column].values
    lower_vals = merged[lower_col].values.astype(float)
    upper_vals = merged[upper_col].values.astype(float)
    inside = (y_true >= lower_vals) & (y_true <= upper_vals)
    coverage = int(np.sum(inside)) / int(len(y_true))
    iqr_vals = upper_vals - lower_vals
    return {
        "coverage": coverage,
        "iqr_mean": float(np.mean(iqr_vals)),
        "iqr_median": float(np.median(iqr_vals)),
        "iqr_std": float(np.std(iqr_vals)),
    }

```